

Session: Training Deep Neural Networks

Javier Serrano

Applied Machine Learning

Master in Data Science and Analytics



Strathmore University

@iLabAfrica Centre

Problems training DNN

Training a deep DNN isn't easy. Some of the problems you could run into are:

1. Vanishing/exploding gradients problem This is when the gradients grow smaller and smaller, or larger and larger, when flowing backward through the DNN during training. Both of these problems make lower layers very hard to train.
2. Not to have enough training data for such a large network, or it might be too costly to label.
3. Training may be extremely slow.
4. A model with millions of parameters would severely risk overfitting the training set, especially if there are not enough training instances or if they are too noisy.

Some Solutions to those problems

1. Vanishing/exploding gradients problem – > **Initialization, activation functions, Batch normalization**
2. Not to have enough training data for such a large network, or it might be too costly to label. – > **Transfer learning**
3. Training may be extremely slow. – > **Faster optimizers: Adam, learning rate scheduling**
4. Overfitting the training set – > **l_1, l_2 regularization, dropout**

Summary

Gradient Problems

Transfer Learning

Faster Optimizers

Avoiding Overfitting

Gradient Problems

- Backpropagation algorithm works by going from the output layer to the input layer, propagating the error gradient along the way.
- Once the algorithm has computed the gradient of the cost function with regard to each parameter in the network, it uses these gradients to update each parameter with a Gradient Descent step.
- Gradients often get smaller and smaller as the algorithm progresses down to the lower layers.
- As a result, the Gradient Descent update leaves the lower layers' connection weights virtually unchanged, and training never converges to a good solution, **vanishing gradients problem**
- In some cases, the opposite can happen: the gradients can grow bigger and bigger until layers get insanely large weight updates and the algorithm diverges, **exploding gradients problem**.

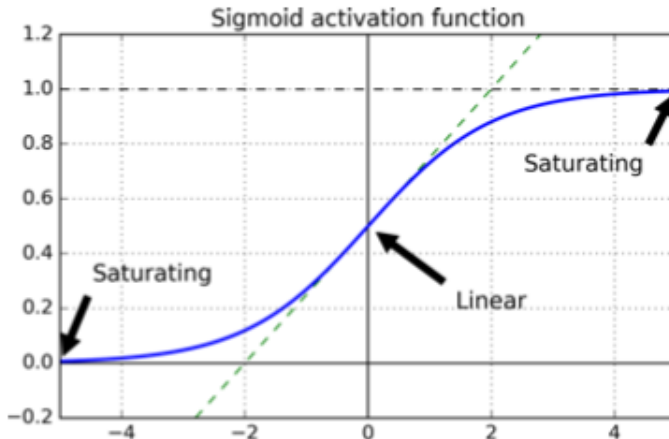
Gradient problems

Solutions

- **Initialization:** Gorot, He, LeCun
- **Activation Functions:** Leaky ReLu, ELU, SELU
- **Batch normalization**

Gradient problems

Logistic activation function saturation



Initialization

Glorot and He Initialization

- To avoid the gradient vanish, we need
 - the variance of the outputs of each layer to be equal to the variance of its inputs
 - the gradients to have equal variance before and after flowing through a layer in the reverse direction
- The following values for a normal distribution to initialize the weight works well

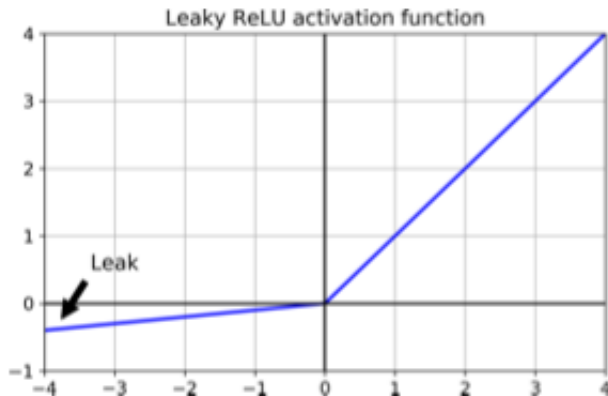
Initialization	Activation functions	σ^2 (Normal)
Glorot	None, tanh, logistic, softmax	$1 / fan_{avg}$
He	ReLU and variants	$2 / fan_{in}$
LeCun	SELU	$1 / fan_{in}$

fan is the number of inputs or output per layer

Better Activation Functions

ReLU

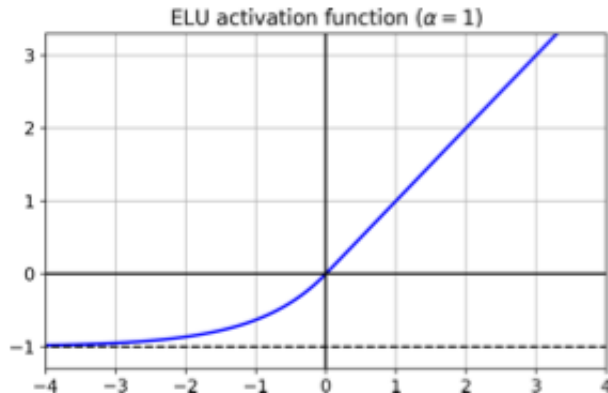
- ReLu is better than a Sigmoid, no saturation
- *Dying* ReLus problem
- Leaky ReLu solve the problem



Better Activation Functions

ELU (Exponential Linear Unit)

- Better than ReLU, has derivative at $z = 0$ and is smooth everywhere helping to speed up the GD (it does not bounce around $z = 0$)



Batch Normalization

Previous methods do not guarantee to avoid the gradient problems during training.

- We add an operation in the model just before or after the activation function of each hidden layer.
- This operation simply zero-centers and normalizes each input
- Then it scales and shifts the result using two new parameter vectors per layer: one for scaling, the other for shifting.
- So, the operation lets the model learn the optimal scale and mean of each of the layer's inputs.

$$1. \quad \mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$$

$$2. \quad \sigma_B^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} (\mathbf{x}^{(i)} - \mu_B)^2$$

$$3. \quad \hat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$4. \quad \mathbf{z}^{(i)} = \gamma \otimes \hat{\mathbf{x}}^{(i)} + \beta$$

Summary

Gradient Problems

Transfer Learning

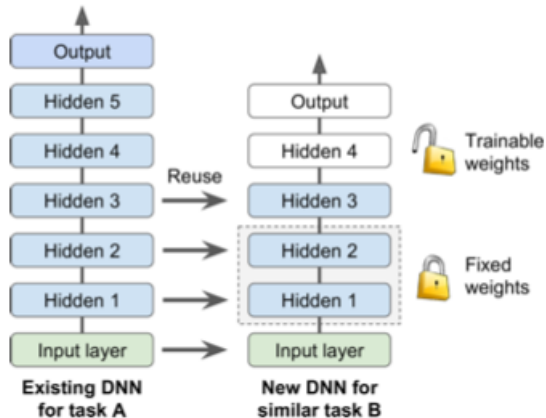
Faster Optimizers

Avoiding Overfitting

Transfer Learning

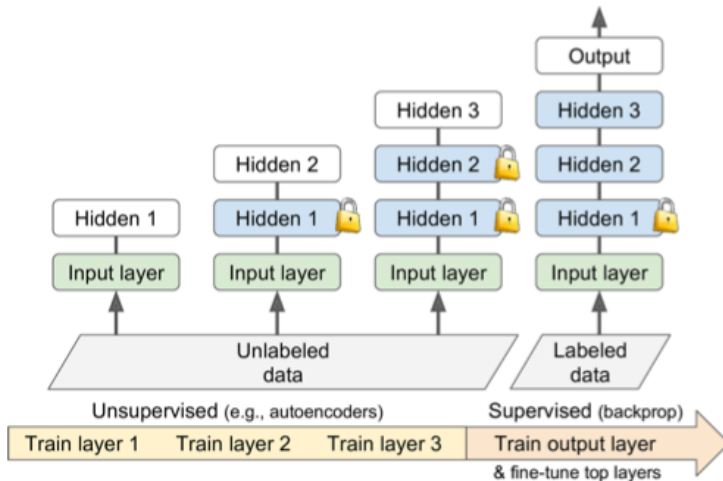
Reusing Pretrained layers

- Generally it is not a good idea to train a very large DNN from scratch
- Try to find an existing neural network that accomplishes a similar task to the one you are trying to tackle
- Reuse the lower layers of this network



Unsupervised Pretraining

Using Autoencoders and GAN



Summary

Gradient Problems

Transfer Learning

Faster Optimizers

Avoiding Overfitting

Faster Optimizer

Momentum Optimization

- Training a very large deep neural network can be very slow.
- Gradient Descent updates the weights $\theta = \theta - \eta \nabla_{\theta} J(\theta)$
- Algorithm ($\beta = 0.9$ is usually a good choice)

$$1. \quad \mathbf{m} \leftarrow \beta \mathbf{m} - \eta \nabla_{\theta} J(\theta)$$

$$2. \quad \theta \leftarrow \theta + \mathbf{m}$$

- The gradient is used for acceleration, not for speed.
- Due to the momentum, the optimizer may overshoot a bit, then come back, overshoot again, and oscillate like this many times before stabilizing at the minimum.

Faster Optimizer

Adam (Adaptative Moment Estimation) Optimization

- Keeps track of an exponentially decaying average of past gradients and squared gradients.
- Algorithm ($\beta_1 = 0.9$ and $\beta_2 = 0.999$ is usually a good choice)

$$1. \quad \mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1 - \beta_1) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

$$2. \quad \mathbf{s} \leftarrow \beta_2 \mathbf{s} + (1 - \beta_2) \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) \otimes \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

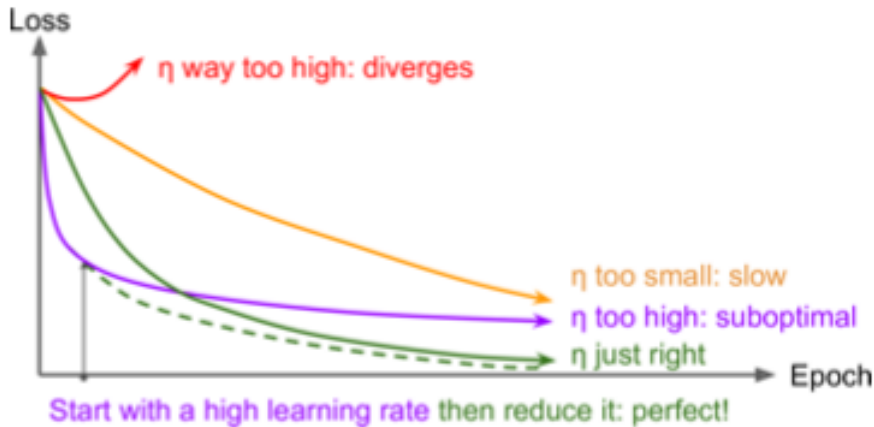
$$3. \quad \widehat{\mathbf{m}} \leftarrow \frac{\mathbf{m}}{1 - \beta_1^t}$$

$$4. \quad \widehat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \beta_2^t}$$

$$5. \quad \boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \eta \widehat{\mathbf{m}} \oslash \sqrt{\widehat{\mathbf{s}} + \epsilon}$$

Faster Optimizer

Learning Rate Scheduling



Summary

Gradient Problems

Transfer Learning

Faster Optimizers

Avoiding Overfitting

Avoiding Overfitting

l_1 and l_2 Regularization

Just like simple linear models, we can use

- l_2 Regularization -> to constrain a neural network's connection weights

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

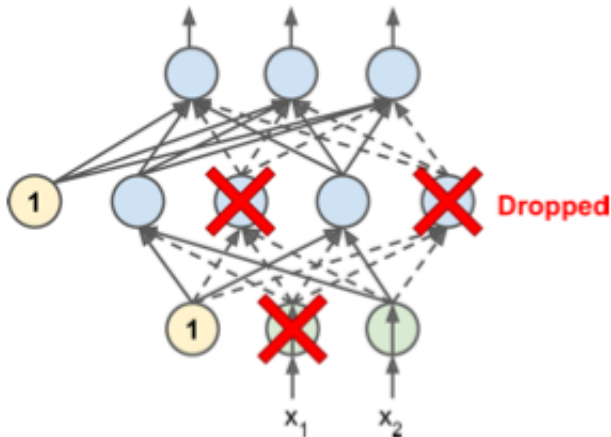
- l_1 Regularization -> sparse model (with many weights equal to 0)

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i|$$

Avoiding Overfitting

Dropout

- At every training step, every neuron (excluding the output neurons) has a probability p of being temporarily “dropped out”.
- The hyperparameter p is called the *dropout rate*, (between 10% and 50%).
- After training, neurons don't get dropped anymore.



Practical Guidelines for choosing methods

Hyperparameter	Default value
Kernel initializer	He initialization
Activation function	ELU
Normalization	None if shallow; Batch Norm if deep
Regularization	Early stopping (+ ℓ_2 reg. if needed)
Optimizer	Momentum optimization (or RMSProp or Nadam)
Learning rate schedule	1cycle

References

- A. Geron, *Hands-on Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2019.
- Andrew Ng course on Deep Learning
<https://www.youtube.com/watch?v=E13qqHb3J7U>
- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, 2016.
<https://github.com/janishar/mit-deep-learning-book-pdf>
- Simon J.D. Prince *Understanding Deep Learning* 2024
<https://udlbook.github.io/udlbook/>